

COMPX304-23A

Advanced Networking and Cybersecurity

Week 9
Operating System Security/vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Stuxnet

An Unprecedented Look at Stuxnet, the World's First Digital Weapon

In an excerpt from her new book, "Countdown to Zero Day," WIRED's Kim Zetter describes the dark path the world's first digital weapon took to reach its target in Iran.



This recent undated satellite image provided by Space Imagery/Inta Space/Funk shows the once-secret Natanz nuclear complex in Natanz, Iran, about 150 miles south of Tehran. AP PHOTO/SPACE IMAGING/INTA SPACE/FUNK. WB



Stuxnet

- Targets industrial control systems
 - Such as those driving centrifuges of nuclear reactors
- Malicious behavior only if the right target is detected
- Rootkit to hide the other two components
- Propagated over an infected USB key



Stuxnet

- How could this happen?
- To answer this, we need to know a few more things:
 - Virtual Address Space Organization/Layout
 - Kernel mode/space vs. User mode/space
 - Operating System (OS) and User Interface
 - Computer Hardware & Device Drivers
 - Driver Signing

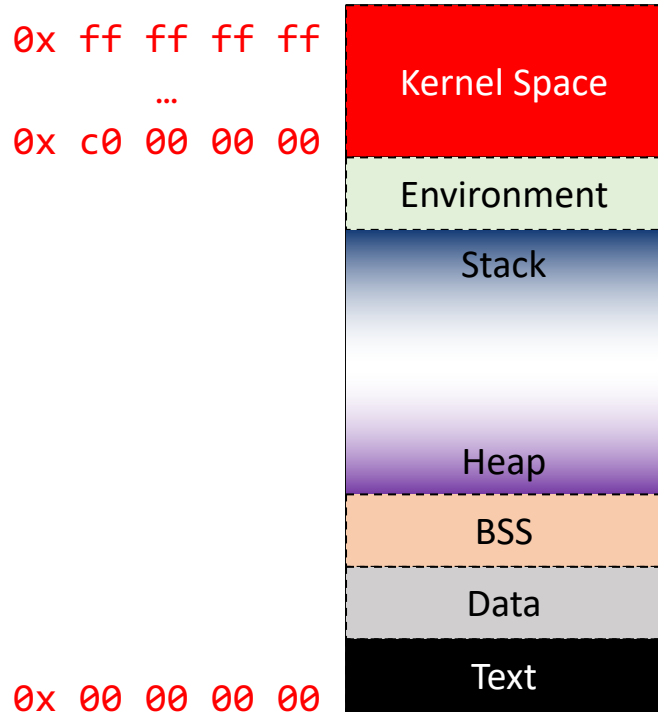


Operating system

Operating system manages and coordinates the use of hardware among the various application programs



Common Memory Layout of Programs

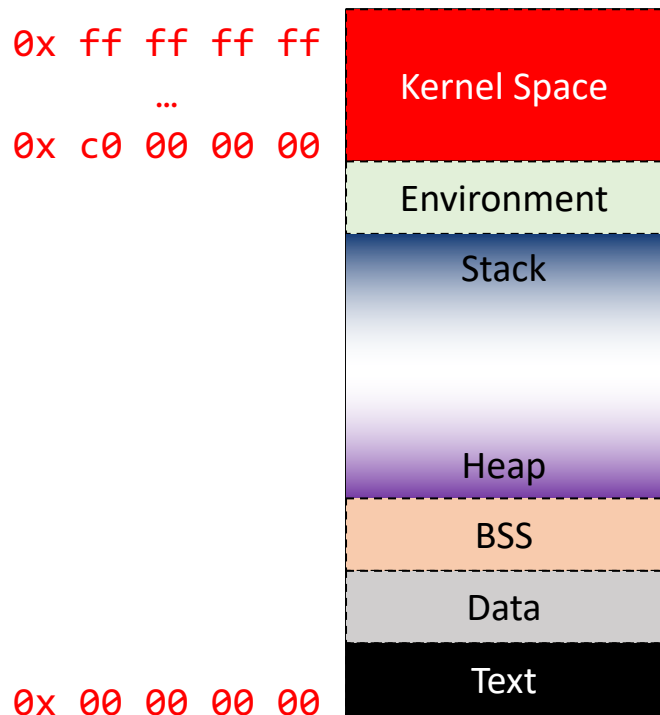


Kernel Space

- This is where the data and code of your OS lives
- Direct access not allowed



Common Memory Layout of Programs



Environment

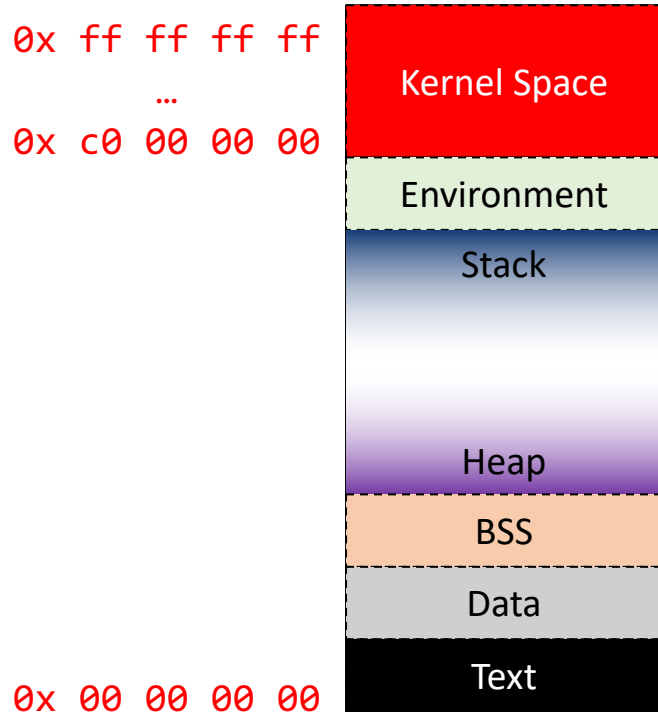
- Command-line arguments
- Environment Variables

```
int main(int argc, char *argv[]){...}
```

```
./myProg 1 2 hello
```



Common Memory Layout of Programs

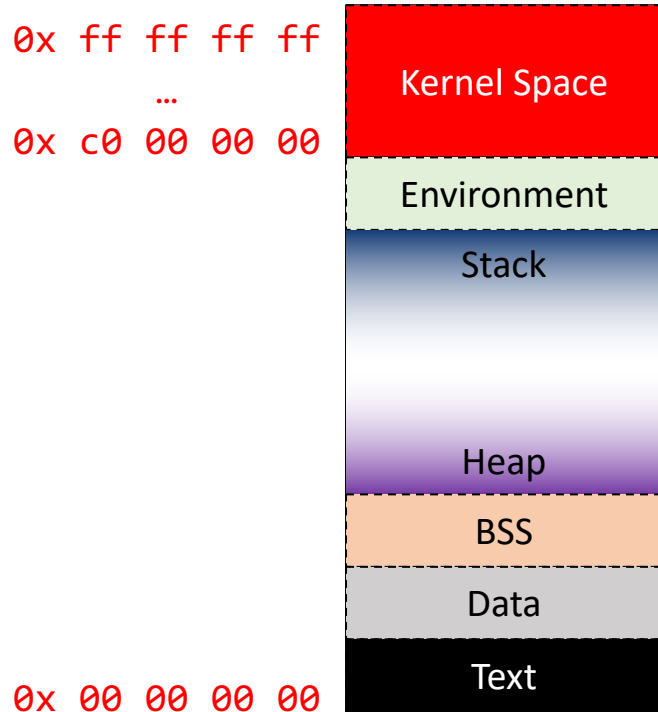


The Stack

- Comprises stack frames
 - One frame per invoked function
 - First one for main function
 - Local variables, return address, return stack pointer
- Stack frames are normally added downwards, towards lower addresses
- What might a stack overflow be?



Common Memory Layout of Programs

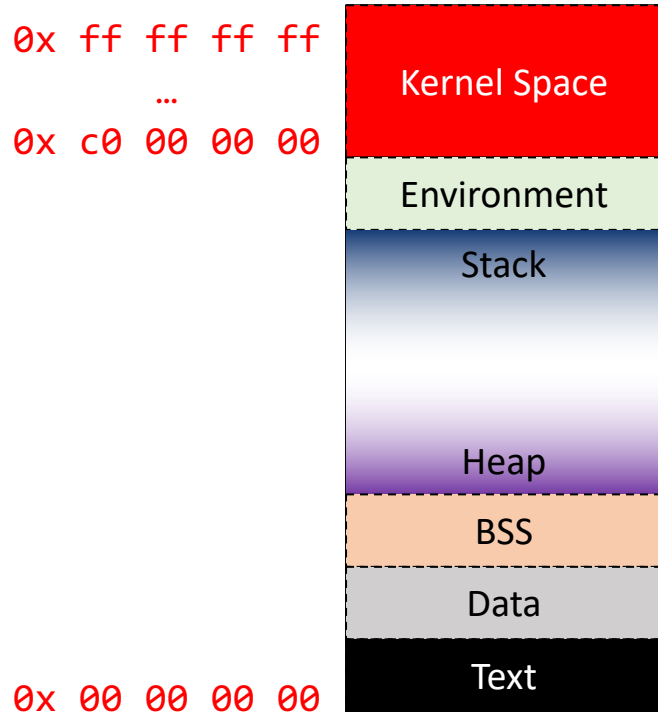


The Heap

- Manual memory management
- Memory chunks allocated by `malloc(size)`
- And are returned with `free(address)`
- What can happen if one allocated too many heap objects?



Common Memory Layout of Programs

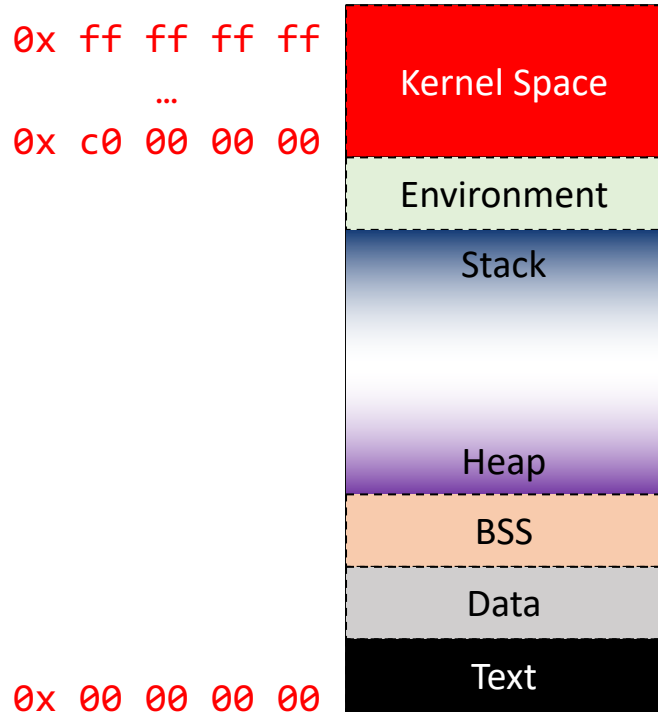


BSS

- Stores uninitialized variables/data
- Historical acronym (Block Started by Symbol)



Common Memory Layout of Programs

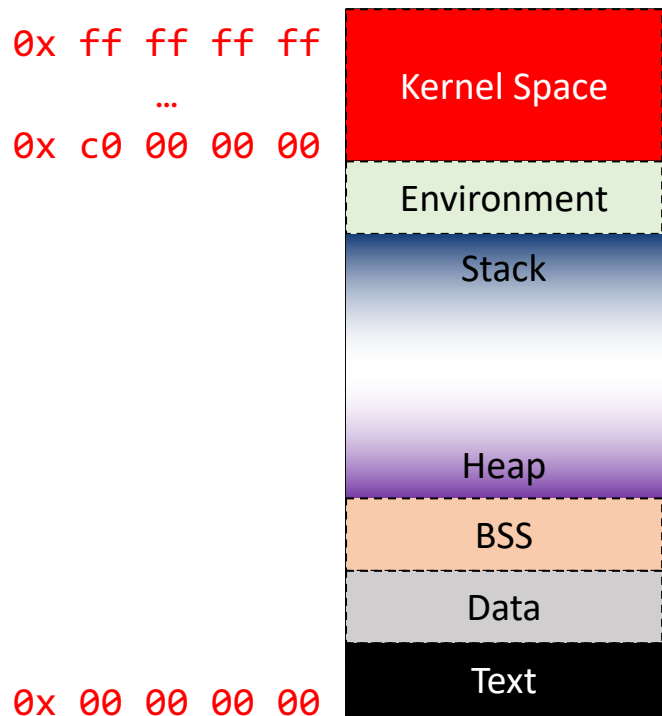


Data

- Stores initialized variables/data



Common Memory Layout of Programs



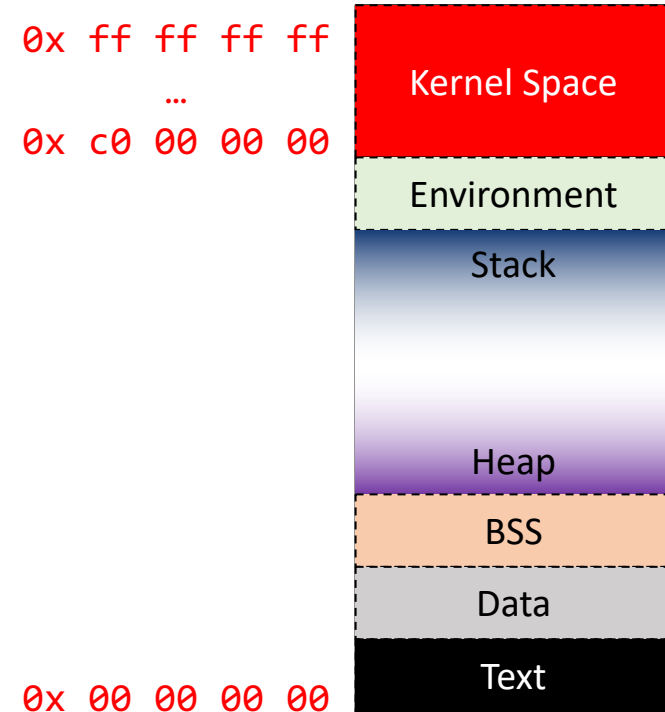
Text

- This is where your compiled code lives!



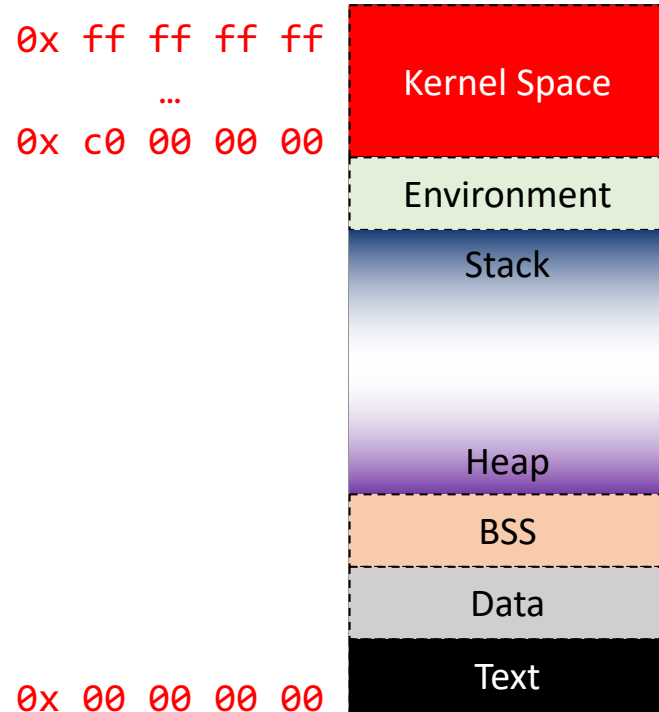
Kernel

- Core of the OS
- Complete control of the system
- Many responsibilities
 - Start up and boot
 - I/O requests
 - Manage tasks, memory, devices
 - Security
 - ...

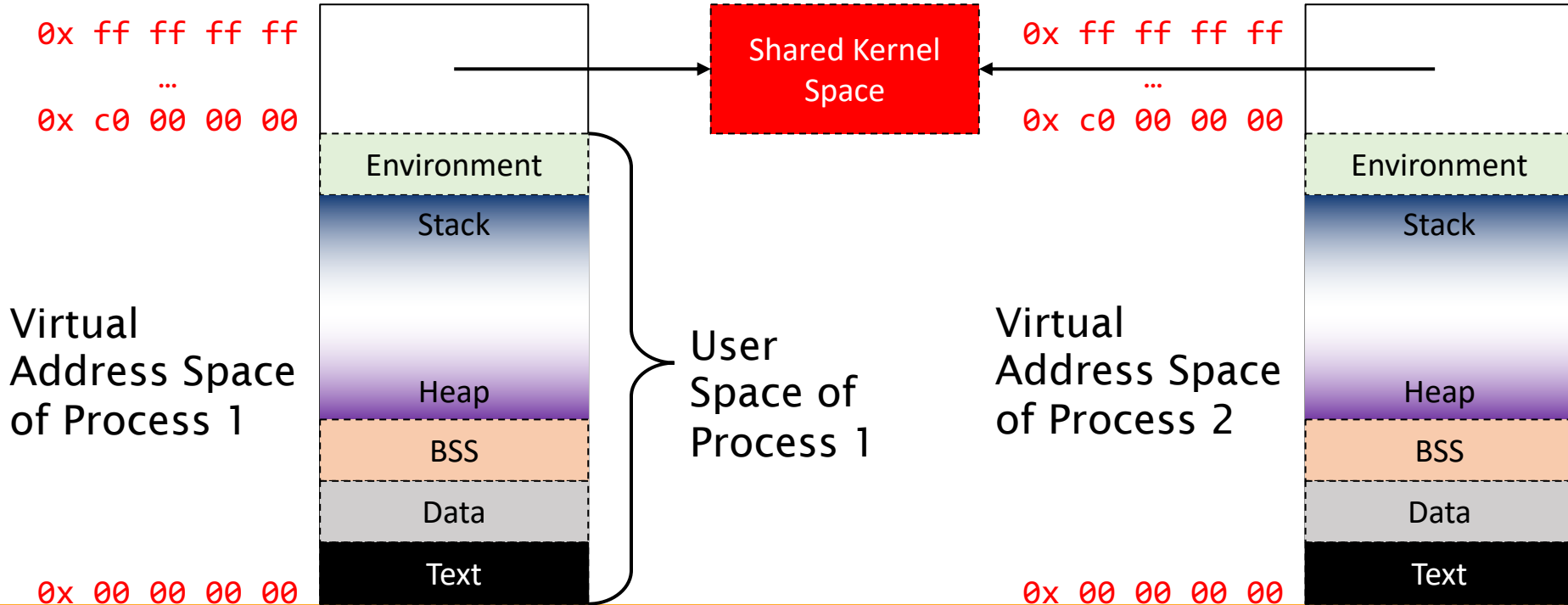


Kernel Space

- The Kernel needs its own memory to store all this
 - Kernel Space
- All running processes share the same kernel space
- But, have their own user space
 - The rest of the memory layout is user space



Kernel Space vs. User Space



CPU Modes and the Kernel

Security is enforced by the hardware!

Kernel Mode

- Any ISA instruction allowed
- Any I/O operation started
- Any memory area accessed

User Mode

- Some ISA instructions restricted
- No direct access to I/O
- Only a small area of the memory is accessible



Invoking the Kernel

- But how can the user invoke the kernel?
- User-mode doesn't have access to Kernel-space



Invoking the Kernel

- But how can the user invoke the kernel?
- Interrupts!
 - The flow of the application stops
 - Execution is transferred to special kernel-address
 - By the hardware! Security needs the hardware!!
 - The Interrupt handler decides what to do based on available information
 - Interrupt handler functions normally use the invoking user's memory space



Invoking the Kernel: System Calls

- An interrupt is too general
 - And doesn't have to be triggered by the user
 - E.g., may handle I/O
- Specific configuration needed for user calling the kernel:
 1. User prepares special registers with:
 - System call identification number
 - Parameters for the system call, including pointers to its user space
 2. User runs a software interrupt (or trap) ISA instruction
 3. Interrupt! Kernel-mode starts!
 4. Interrupt-handler realizes it's a system call
 5. It invokes it by looking at the special registers



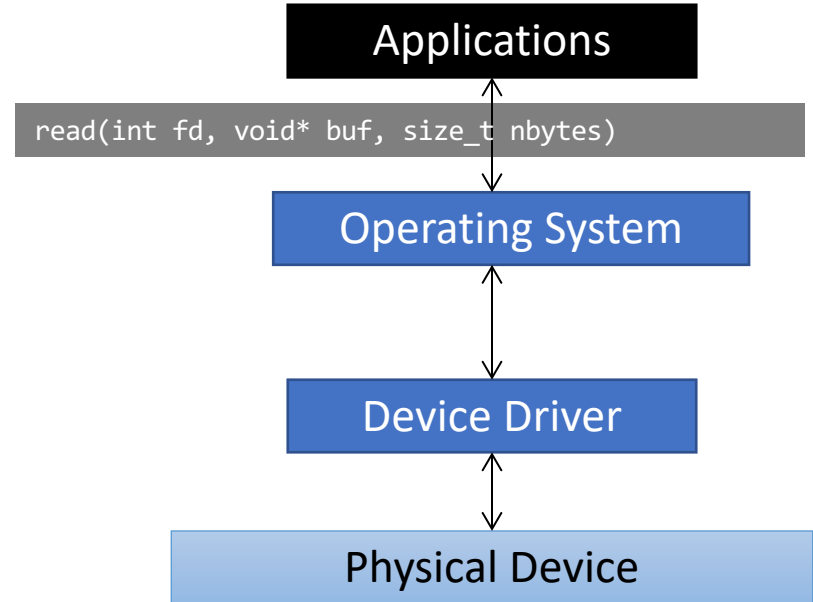
Kernel-space Information: EXTREMELY SENSITIVE!!!

- **Task Structures:** Tasks (processes, apps) activated
- **Process Table:** Points to Task Structures
- **Files and Inodes:** Open files
- **Memory management:** How the physical memory is allocated
- **Wait Queues:** What tasks wait on some resource
- **Kernel Semaphores:** Synchronize the tasks of the kernel
- **System Timers:** Used for scheduling, etc.



Computer Hardware and the OS

- Everything is a file in Linux
- The OS offers certain system calls:
 - Read, Write, Open, Close, Etc.
- Interacting with a device is not the same for all devices
- Each device needs to be driven according to its own specs
- Device Drivers!



Device Drivers

- Abstract control of a device
- Device-dependent
- Operating-System-specific
- Quote:

“Device drivers are a collection of routines which write magical numbers to magical places in the hardware”

- Security
 - Run in Kernel-Mode
 - Static or Dynamic Loading
 - Use Kernel-Space memory
- Otherwise
 - User apps would have direct access to:
 - See others' IO
 - Change others IO
 - Break/stall/etc. the device



Drivers

- Not all drivers are device drivers
- Some drivers do run in user-space
 - Really not common
- On Windows, GPU drivers are a special case
 - This is an aside 😊



But how can we just give access to a driver??

- Software running in kernel-mode can do:
 - ANYTHING!!
- Read and modify your data
- Install and/or start apps
- Uninstall and/or stop apps
- Crash your system (Unless it's a graphics driver)
- Wait and bide its time until something happens... 😞



Driver Signing

- Association of a digital signature
- With a specific driver package
- A driver package must be verified before loaded into/by the kernel



Digital Signature

1. Alice encrypts her message with her private key
 $DS = \text{Encrypt}(M, A_{\text{private}})$
2. Alice sends both messages to Bob
 $\text{Send}(\text{Bob}, \langle M, DS \rangle)$

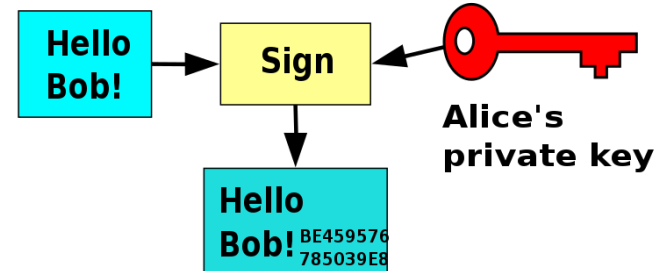
3. Bob decrypts the digital signature with Alice's public key

$$M' = \text{Decrypt}(DS, A_{\text{public}})$$

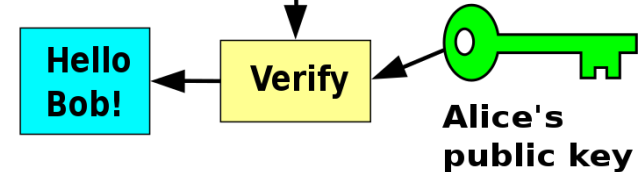
4. Bob compares the decrypted and openly submitted message to confirm authenticity

$$M = M'?$$

Alice



Bob



https://en.wikipedia.org/wiki/Digital_signature



Driver Signing

- Device manufacturers have private keys. They distribute the corresponding public keys to the Operating Systems manufacturers
- When a new device driver software is written the device manufacturer signs it with their private key
- If the Operating System is able to verify a software using one of the public keys it has, it will trust the software and use it



Stuxnet

- The hackers had stolen private keys from two reputable companies
 - Realtek and JMicron
- Used them to sign their malicious “drivers”
- The OS trusted them and installed them...!
- Once in the kernel:
 - It rapidly copied itself and propagated to other machines
 - Through other local-network vulnerabilities
 - It remained dormant unless it detected a specific Siemens control system
- Also, two websites were used to further control and patch the worm
 - Command and Control (C2)

